Somaiya Vidyavihar University · K J Somaiya School of Engineering

Cloud Computing · Mini Project Report

HeatGuard: 7-day Heatwave Forecast and Early Warning

Cloud Computing Experiments integrated with the Somaiya AI Use Case
KJS-CES-01: Climate Intelligence for Heatwave Monitoring, Prediction and Early Warning

Student	**Ayushi Ranjan**
Roll No.	16014223028
Batch	B1
Submission	Individual
Selected use case	Case 5: Climate Intelligence for Heatwave Monitoring, Prediction and Early Warning
Selected experiments	**2** Containerization, Docker and Docker Hub · **3** Application Deployment using Kubernetes · **7** Cloud Cost Optimization through Resource Right-Sizing · **8** FinOps Principles and Instance-Type Comparison
Source code	https://github.com/ayushiii2707/CC-Mini-Project---Heatwave-Forecast
Docker images	https://hub.docker.com/u/ayushiranjan27 (heatguard, heatguard-web)
Demonstration video	[Placeholder: demonstration video link, to be inserted after upload]
Date	11 October 2026

Abstract

HeatGuard is a working prototype of the heatwave early-warning use case. It uses real India Meteorological Department (IMD) gridded daily maximum-temperature data for 2014–2024, forecasts the maximum temperature for each of the next **7 days** for ten Indian cities with machine-learning models, and converts every forecast day into a heatwave alert using IMD's published criteria. On the unseen year 2024 the forecast error grows from **0.883 °C** (day 1) to **1.542 °C** (day 7), lower than both baselines at every horizon. Next-day alerts detect **90%** of actual heatwave city-days; alerts for days 5–7 are marked low-confidence because validation showed limited skill. The system is split into a FastAPI **backend** and an nginx-served **web frontend**, both published on Docker Hub (Experiment 2) and deployed on Kubernetes with scaling, self-healing, rollback and measured rolling updates (Experiment 3). The backend was right-sized from 2 vCPU / 2 GB to 1.25 vCPU / 192m with an estimated **-42.7%** change in hourly cost (Experiment 7), and three instance sizes were compared under the same workload against a target fixed in advance (Experiment 8). All results come from real runs; costs are estimates.

1. Problem statement, objectives and scope

Problem. Heatwaves in India are increasing in frequency, intensity and duration and affect public health, agriculture and economic activity. Severity varies by region, so timely, region-specific warnings are needed (use-case document KJS-CES-01, §6).

Objectives.

1. Build an input → processing → output prototype on real IMD meteorological data (use case §11, steps 1–2).
2. Develop AI models for region-wise maximum-temperature forecasting up to 7 days ahead and evaluate them against baselines (§8, objective 2).
3. Classify heatwave severity per day, produce advisories and visualise them on a decision-support dashboard (§8, objective 4; §11, steps 7–9).
4. Package, publish, deploy, measure and cost-optimise the service using Experiments 2, 3, 7 and 8.

Prototype scope.

In scope	Out of scope (see Limitations)
IMD 1°×1° gridded daily Tmax 2014–2024, ten city grid cells	IoT Automated Weather Stations (hardware)
7-day Tmax forecast (one ML model per day) + IMD rule-based severity	Humidity/heat-index inputs, numerical weather model input
Web frontend: alerts, 7-day cards, trend chart, India map, ranking, what-if	LLM-generated multilingual advisories (future work)
Backend REST API, Docker, Docker Compose, Kubernetes	Real cloud VM deployment and billing (costs are estimates)

2. Dataset

- **Source:** IMD Pune, "High resolution 1°×1° daily gridded maximum temperature" binary data, imdpune.gov.in/cmpg/Griddata/Max_1_Bin.html (Srivastava, Rajeevan and Kshirsagar, 2009), the

repository named in the use-case document (§11, step 1).

- **Format:** one `.GRD` file per year, float32, 31 × 31 grid (7.5–37.5 °N, 67.5–97.5 °E), 99.9 = missing. File sizes were checked (1,403,060 bytes for 365-day years, 1,406,904 bytes for leap years).
- **Acquisition:** `scripts/download_imd.sh`. The IMD server is slow and dropped connections (2017 needed three attempts), so the script retries and accepts only full-size files.
- **Pre-processing:** `scripts/prepare_data.py` extracts the grid cell nearest to each of ten cities (Delhi, Jaipur, Lucknow, Patna, Ahmedabad, Nagpur, Kolkata, Bhubaneswar, Hyderabad, Chennai): 40,180 daily rows. Grid orientation was verified (sea cells missing, Ladakh cold, Delhi ≈ 17 °C in January and ≈ 40 °C in May).
- **Split (time-ordered by the forecast target day):** train 2014–2022, validation 2023, test 2024. Day-1 samples: train 32800, validation 3650, test 3660.
- **No synthetic data** is used. The data is non-personal meteorological data.

3. System architecture and tools

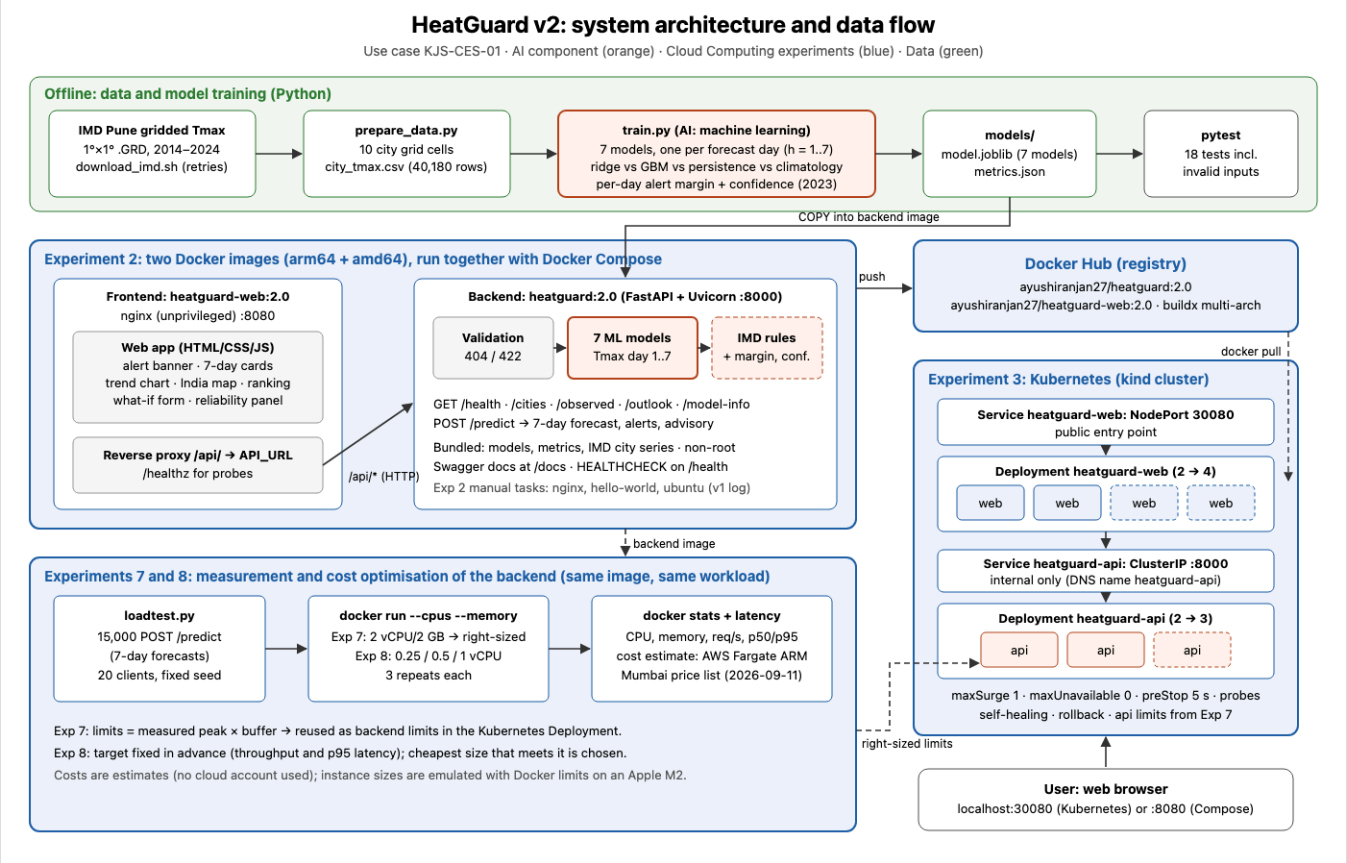


Figure 1. Labelled architecture and data flow (v2). Orange: AI component. Blue: Cloud Computing experiments. Green: data pipeline.

3.1 Why an early-warning service needs these cloud capabilities

A heatwave warning is most valuable exactly when demand peaks: during a hot spell many people open the dashboard at once, and the service cannot be down while it is being updated. Packaging the backend and frontend as separate images (Exp 2) means each can be rebuilt, versioned and moved to any machine unchanged. Running them on Kubernetes (Exp 3) adds replicas, health probes and self-healing, so a crashed pod does not take the warning offline, and rolling updates let a retrained model be released without an outage. Because such a service runs all year but is busy only in summer, paying for idle capacity is the

main waste; Exp 7 measures how much of the allocated capacity is really used, and Exp 8 picks the cheapest size that still meets a stated response-time and throughput target.

3.2 Tools

Layer	Tools (versions as run)
Language / ML	Python 3.13, NumPy 2.5.4, scikit-learn 1.9.1, pandas 3.0.6 (training only), joblib 1.6.0
Backend	FastAPI 0.143.0, Uvicorn 0.54.0 (image python:3.13-slim, non-root)
Frontend	HTML/CSS/JavaScript (landing page + dashboard), Chart.js 4.4.1, Leaflet 1.9.4, Natural Earth boundaries (India view), bundled fonts Fraunces / IBM Plex (SIL OFL), served by nginx 1.27 (nginx-unprivileged)
Testing / load	pytest 9.1.1, httpx 0.28.1 (async load generator)
Containers (Exp 2)	Docker Desktop engine 29.7.2 (arm64), Docker Compose, docker buildx, Docker Hub
Orchestration (Exp 3)	kind v0.33.0 (Kubernetes v1.37.0), kubectl v1.36.1
Measurement (Exp 7, 8)	docker stats, experiments/run_config.py, matplotlib 3.11.2
Hardware	Apple MacBook M2, 8 GB RAM (Docker VM: 8 CPUs, 3.8 GiB)

4. AI component

The AI part is kept separate from the cloud part so each can be explained on its own.

4.1 Machine-learning forecaster (7 days ahead)

Method: direct multi-horizon forecasting. Seven separate models are trained; model *h* predicts the maximum temperature on day *t + h* from the 7 days up to day *t*. **Features** (`app/features.py`, shared by training and serving): the last 7 daily Tmax values, their mean, the target day's season (sine/cosine of day-of-year) and climatological normal (from 2014–2022 only), and the cell's latitude and longitude. Two model families (ridge regression, gradient boosting) were compared with two baselines: *persistence* (same as today) and *climatology* (the normal for that date). The family with the lowest mean validation error (ridge_regression, 1.516 °C) was deployed; 2024 was used only once, for the final numbers below.

4.2 Rule-based severity classifier (not ML)

`app/severity.py` applies the IMD heatwave definitions: a heatwave is considered only if Tmax ≥ 40 °C (plains) or ≥ 37 °C (coastal). Departure from normal 4.5–6.4 °C → Heatwave, > 6.4 °C → Severe heatwave; for plains, Tmax ≥ 45 °C → Heatwave and ≥ 47 °C → Severe. Coastal stations (Kolkata, Chennai) have only a Heatwave category.

4.3 Alert margin and confidence per forecast day

Regression pulls extreme values towards the mean, so a raw forecast rarely crosses the IMD thresholds (day 1 without a margin: recall 29%). Because a missed heatwave is costlier than a false alarm, each day's alert is issued on an **upper-bound forecast** (forecast + margin). The margin for each day was chosen on the 2023 validation year by F2 score (recall weighted over precision). Where the best validation F2 was

below 0.3, that day's alerts are labelled **low-confidence** in the API and the dashboard. Validation contains only 14 heatwave city-days, so these choices are uncertain.

Day	Persistence MAE	Climatology MAE	Grad. boosting MAE	ridge_regression MAE (deployed)	Alert margin	Val F2	Test recall	Test precision	Alert confidence
Day 1	0.94	1.575	0.902	**0.883**	1.25 °C	0.618	0.896	0.566	validated
Day 2	1.379	1.575	1.262	**1.235**	2.75 °C	0.38	0.979	0.299	validated
Day 3	1.652	1.575	1.438	**1.396**	3.0 °C	0.388	0.854	0.295	validated
Day 4	1.84	1.575	1.536	**1.475**	3.25 °C	0.381	0.729	0.257	validated
Day 5	1.976	1.575	1.562	**1.514**	3.75 °C	0.276	0.771	0.164	**low**
Day 6	2.086	1.575	1.586	**1.534**	3.25 °C	0.26	0.354	0.23	**low**
Day 7	2.155	1.575	1.587	**1.542**	3.5 °C	0.294	0.396	0.184	**low**

All MAE values are in °C on the 2024 test year (48 actual heatwave city-days per horizon). **Interpretation:** the ML model beats persistence at every horizon and by a widening margin; by day 7 it is only slightly better than climatology, which is expected for weather forecasting from temperature history alone. Alert skill falls with lead time: days 1–4 are validated; days 5–7 catch fewer heatwaves and raise more false alarms, so they are shown as an outlook. Day-1 confusion matrix (rows = actual, columns = predicted; Normal / Heatwave / Severe): [[3579, 33, 0], [5, 37, 2], [0, 3, 1]]. Persistence baseline day-1 alert recall/precision: 0.688 / 0.647.

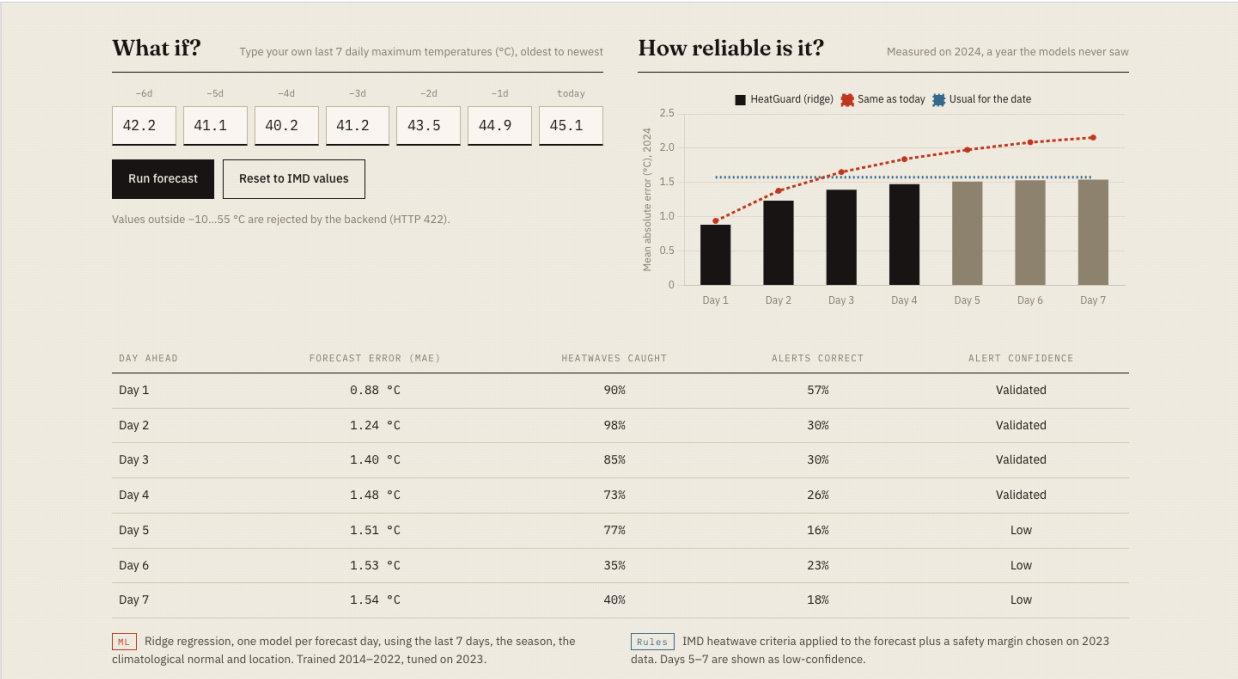


Figure 2. The dashboard's reliability panel: forecast error by day (ML vs baselines) and alert skill per day, read live from the API.

5. Application: backend and frontend

Backend (`app/main.py`, image `ayushiranjan27/heatguard:2.0`): a stateless FastAPI service. Endpoints: `/health`, `/cities`, `/observed` (IMD history with normals and the actual next 7 days), `/outlook` (7-day forecast for all 10 cities), `/model-info` (per-horizon metrics) and `POST /predict` (7-day forecast from 7 values). Input validation returns 404/422 with clear messages; interactive API docs are at `/docs`. **Frontend** (`frontend/`, image `ayushiranjan27/heatguard-web:2.1`): a static web app served by unprivileged nginx, which also reverse-proxies `/api/` to the backend (and the backend's interactive docs at `/api/docs`), so the browser talks to one address and the backend can stay internal. It has two pages with one visual identity (a printed "heat bulletin": paper and ink for the interface, heat colours only for data, bundled fonts so nothing depends on the internet):

- **Landing page** (`/`): an animated introduction whose figures all come from the live API: a hero card with the highest-risk city for 27 May 2024, a "heat stripes" band of every 2024 IMD daily maximum for Delhi, count-up key figures read from `/model-info`, a scroll-revealed explanation of the pipeline, a pulsing India map of the 10-city outlook, a per-day accuracy chart, and the four cloud experiments with their measured results. Animations respect the operating system's reduced-motion setting.
- **Dashboard** (`/dashboard`): the 7-day heat bulletin for any city and date in 2014–2024: headline alert and advice, KPI row, a seven-day forecast table (forecast ± typical error, alert, alert basis, departure, the actual IMD value, confidence), a 30-day trend chart with uncertainty band, an India map and ranking, a what-if form, and the reliability panel.

The map uses bundled Natural Earth boundaries (India point-of-view edition), so it works without an internet map service. Version 2.0 of the frontend (a single dark dashboard) is kept in the archive; 2.1 changed only the frontend, so the backend and all measurements are unchanged.

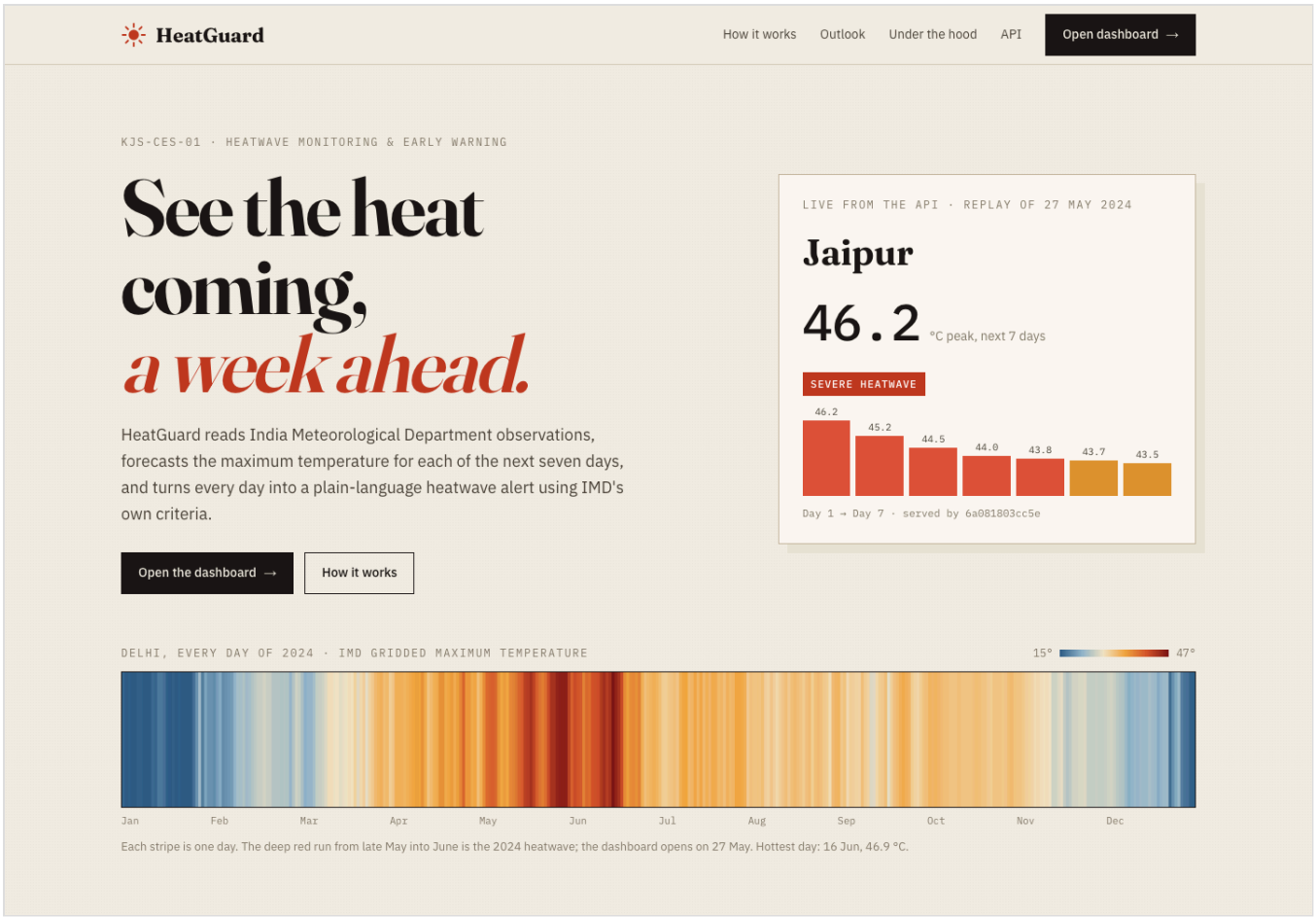


Figure 3. Landing page: animated headline, live hero card (highest-risk city on 27 May 2024 from /outlook) and the 2024 Delhi heat stripes from IMD data.

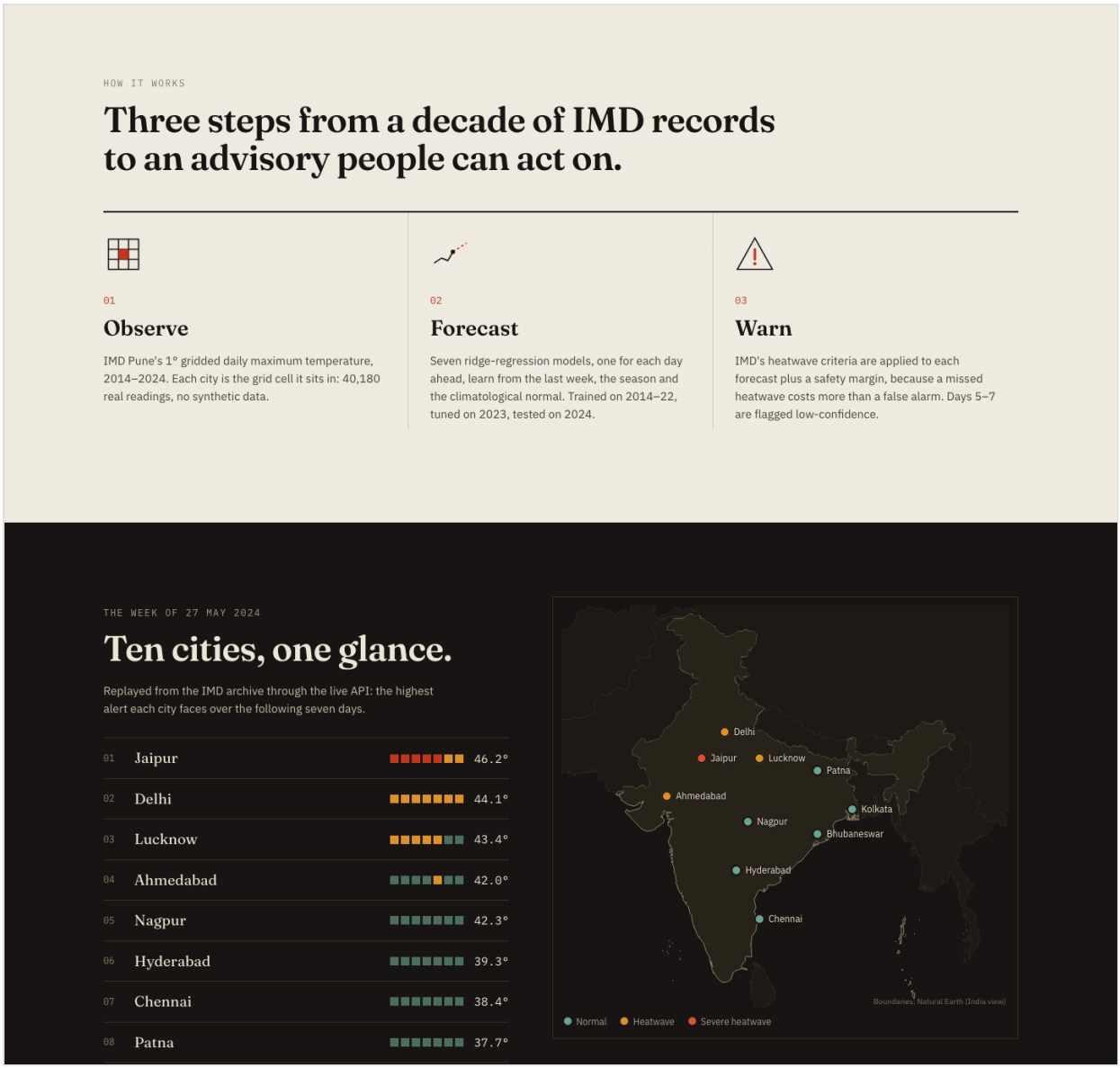


Figure 4. Landing page: how it works, and the 10-city outlook with a pulsing India map (Natural Earth, India view).

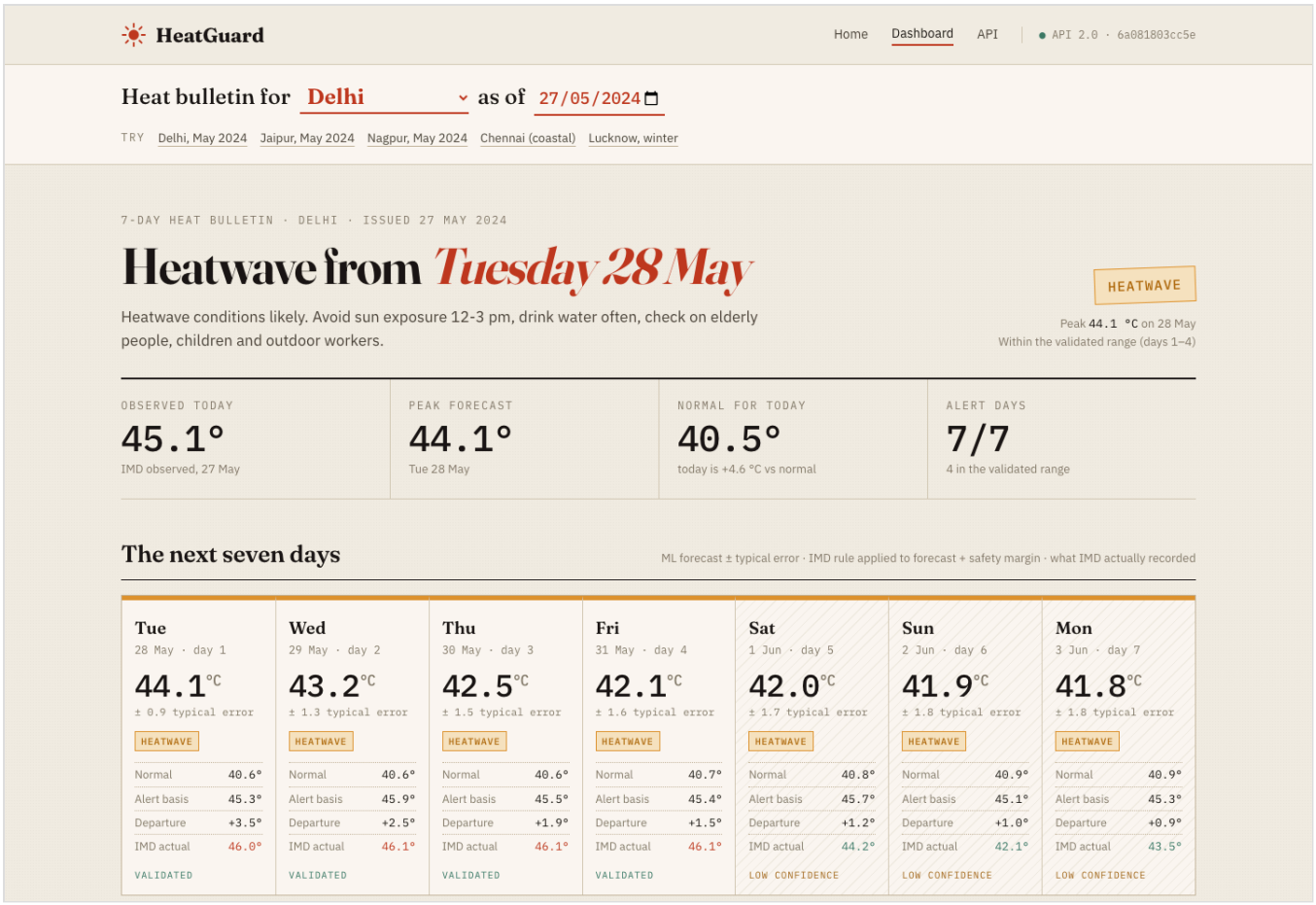


Figure 5. Dashboard, Delhi as of 27 May 2024 (served by Docker Compose): bulletin headline, KPIs and the 7-day forecast table with the actual IMD values that followed.

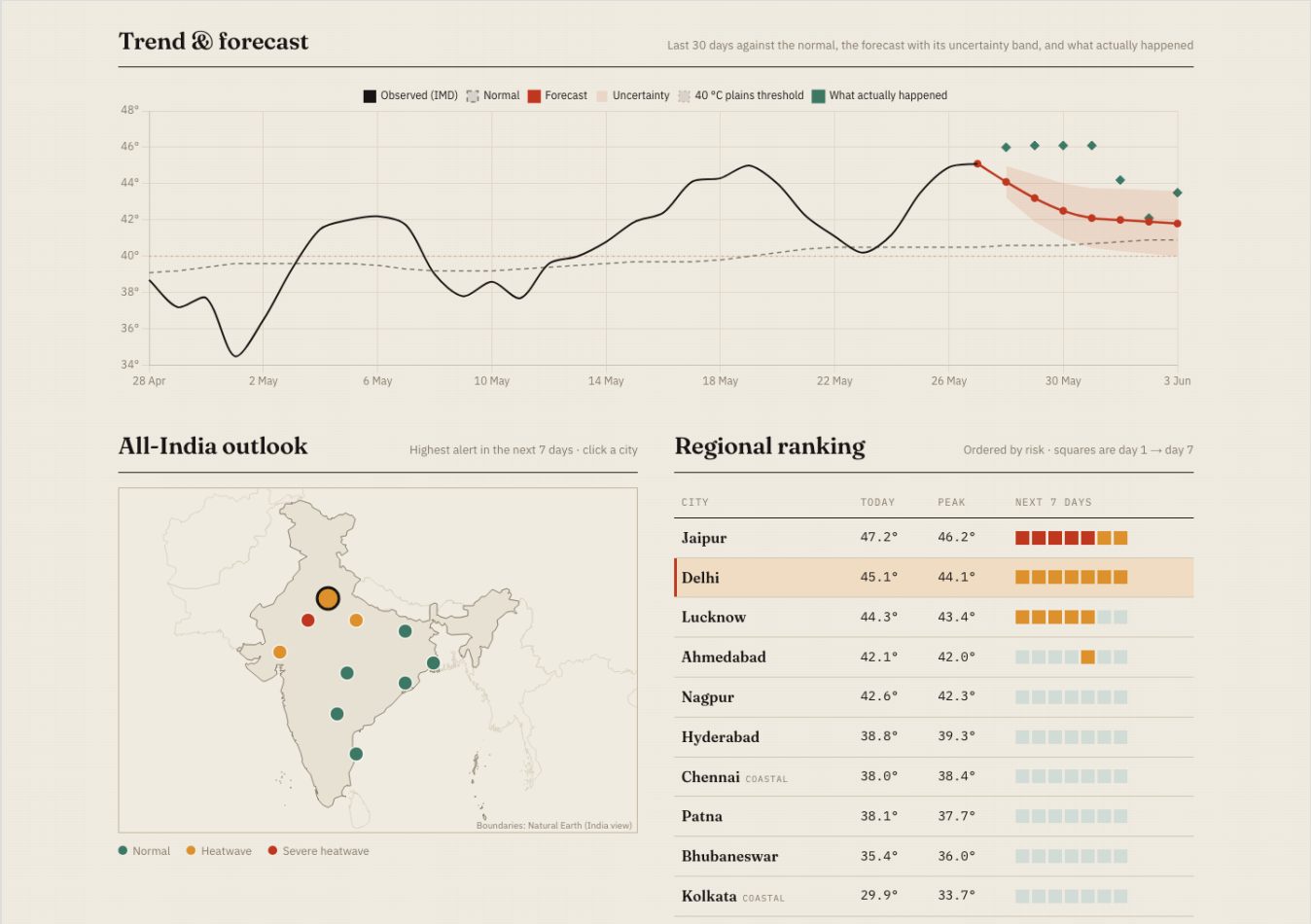


Figure 6. Dashboard: trend chart (observed, normal, forecast with uncertainty band, actual IMD values), all-India map and regional ranking.

6. Implementation of the four experiments

Experiment 2: Containerization, Docker and Docker Hub

Aim (manual): install Docker, pull and run images from Docker Hub, modify containers to run an application and push it to Docker Hub. **Use-case connection:** the forecasting backend (code + 7 trained models + IMD city data) and the web frontend are packaged as two images, which every later experiment uses unchanged.

Procedure. (1) Docker Desktop installed; Docker Hub account `ayushiranjan27` created and `docker login` done by the student. (2) Manual tasks: nginx pulled and run on port 8080, hello-world and Ubuntu run, Docker commands explored (`experiments/results/archive_v1/exp2_manual_tasks.log`; nginx was pushed as `ayushiranjan27/nginx-exp2:1.0`). (3) Two Dockerfiles and a Compose file written. (4) Both images built and run together, tested through the frontend proxy, including failure cases. (5) Both pushed for arm64 and amd64 with `docker buildx`. (6) Local images deleted, pulled back from Docker Hub and run. Script: `experiments/exp2_docker.sh`.

Dockerfile

```
# HeatGuard backend: 7-day heatwave forecast API (Experiment 2: containerisation)
FROM python:3.13-slim

WORKDIR /srv
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Application code, trained model and the processed IMD city series used by /observed
COPY app/ app/
COPY models/ models/
COPY data/processed/ data/processed/

# Run as an unprivileged user
RUN useradd --create-home appuser
USER appuser

EXPOSE 8000
HEALTHCHECK --interval=15s --timeout=3s --start-period=10s \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

frontend/Dockerfile

```
# HeatGuard web frontend (landing page + dashboard): static app served by nginx (unprivileged, port 8080),
# with /api/ reverse-proxied to the backend given by API_URL.
FROM nginxinc/nginx-unprivileged:1.27-alpine
ENV API_URL=http://api:8000
COPY nginx/default.conf.template /etc/nginx/templates/default.conf.template
COPY public/ /usr/share/nginx/html/
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=3s --start-period=5s \
  CMD wget -qO- http://127.0.0.1:8080/healthz || exit 1
```

docker-compose.yml

```
# Run HeatGuard v2 locally as two containers: backend API + web frontend.
#   docker compose up -d   ->  open http://localhost:8080
name: heatguard
services:
  api:
    image: ayushiranjan27/heatguard:2.0
    build: .
    ports: ["8000:8000"]          # direct API access (Swagger docs at http://localhost:8000/docs)
  web:
    image: ayushiranjan27/heatguard-web:2.1
    build: ./frontend
    environment:
      API_URL: http://api:8000
    ports: ["8080:8080"]
    depends_on:
      api:
        condition: service_healthy
```

experiments/results/exp2_docker.log

```
### 2. Run backend + frontend together (docker-compose.yml)
$ docker compose up -d --wait
 Network heatguard_default Creating
 Network heatguard_default Creating
 Network heatguard_default Created
 Network heatguard_default Created
 Container heatguard-api-1 Creating
 Container heatguard-api-1 Created
 Container heatguard-web-1 Creating
 Container heatguard-web-1 Created
 Container heatguard-api-1 Starting
 Container heatguard-api-1 Started
 Container heatguard-api-1 Waiting
 Container heatguard-api-1 Healthy
 Container heatguard-web-1 Starting
 Container heatguard-web-1 Started
 Container heatguard-web-1 Waiting
 Container heatguard-api-1 Waiting
 Container heatguard-api-1 Healthy
 Container heatguard-web-1 Healthy

$ docker compose ps
NAME              IMAGE                              COMMAND                  SERVICE   CREATED          STATUS
PORTS
heatguard-api-1   ayushiranjan27/heatguard:2.0       "uvicorn app.main:ap…"   api       11 seconds ago   Up 11 se
conds (healthy)   0.0.0.0:8000->8000/
heatguard-web-1   ayushiranjan27/heatguard-web:2.0   "/docker-entrypoint.…"   web       11 seconds ago   Up 5 sec
onds (healthy)    0.0.0.0:8080->8080/

$ curl -s localhost:8000/health
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"15f51d4e40e3"}
$ curl -s localhost:8080/healthz
ok

$ curl -s localhost:8080/api/health          # frontend -> nginx proxy -> backend
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"15f51d4e40e3"}

$ curl -s -X POST localhost:8080/api/predict (Delhi, real IMD values 21-27 May 2024)
highest alert: Heatwave | validated: Heatwave | first alert day: 1 | served by 15f51d4e40e3 in 16.85 ms
```

experiments/results/exp2_docker.log

```
### 3. Failure scenarios (through the frontend proxy)
# unknown city
{"detail":"unknown city 'Atlantis'. Supported: Delhi, Jaipur, Lucknow, Patna, Ahmedabad, Nagpur, Kolkata, Bhubane
swar, Hyderabad, Chennai"} HTTP 404
# 99.9 = IMD missing code
{"detail":[{"type":"value_error","loc":["body","recent_tmax"],"msg":"Value error, temperature 99.9 C is outside t
he plausible range -10..55 C","input"
# date outside the data
{"detail":"no IMD observations for Delhi for the 7 days up to 2030-05-01 (available: 2014-01-07 to 2024-12-31)"}
HTTP 404
# backend stopped -> frontend stays up, API calls fail with a gateway error
  /healthz HTTP 200   /api/health HTTP 504
# backend restarted -> /api/health HTTP 200
```

experiments/results/exp2_docker.log

```
### 4. Push both images to Docker Hub for linux/arm64 and linux/amd64
$ docker buildx build --platform linux/arm64,linux/amd64 -t ayushiranjan27/heatguard:2.0 -t ayushiranjan27/heatgu
ard:latest --push --quiet .

$ docker buildx build --platform linux/arm64,linux/amd64 -t ayushiranjan27/heatguard-web:2.0 -t ayushiranjan27/he
atguard-web:latest --push --quiet ./f

$ docker buildx imagetools inspect ayushiranjan27/heatguard:2.0 | grep Platform
  Platform:    linux/arm64
  Platform:    linux/amd64

$ docker buildx imagetools inspect ayushiranjan27/heatguard-web:2.0 | grep Platform
  Platform:    linux/arm64
  Platform:    linux/amd64
```

experiments/results/exp2_docker.log

```
$ docker rmi ayushiranjan27/heatguard:2.0 ayushiranjan27/heatguard-web:2.0
Untagged: ayushiranjan27/heatguard:2.0
Untagged: ayushiranjan27/heatguard-web:2.0

$ docker compose pull
 Image ayushiranjan27/heatguard:2.0 Pulling
 Image ayushiranjan27/heatguard-web:2.0 Pulling
 Image ayushiranjan27/heatguard-web:2.0 Pulled
 Image ayushiranjan27/heatguard:2.0 Pulled

$ docker compose up -d --wait --no-build
 Network heatguard_default Creating
 Network heatguard_default Creating
 Network heatguard_default Created
 Network heatguard_default Created
 Container heatguard-api-1 Creating
 Container heatguard-api-1 Created
 Container heatguard-web-1 Creating
 Container heatguard-web-1 Created
 Container heatguard-api-1 Starting
```

The redesigned frontend was later released as `heatguard-web:2.1` with the same tooling (`experiments/release_frontend.sh`):

experiments/results/frontend_2.1_release.log

```
### 1. Build and push ayushiranjan27/heatguard-web:2.1 (linux/arm64 + linux/amd64)
$ docker buildx build --platform linux/arm64,linux/amd64 -t ayushiranjan27/heatguard-web:2.1 -t ayushiranjan27/he
atguard-web:latest --push --quiet ./f

$ docker buildx imagetools inspect ayushiranjan27/heatguard-web:2.1 | grep Platform
  Platform:    linux/arm64
  Platform:    linux/amd64
```

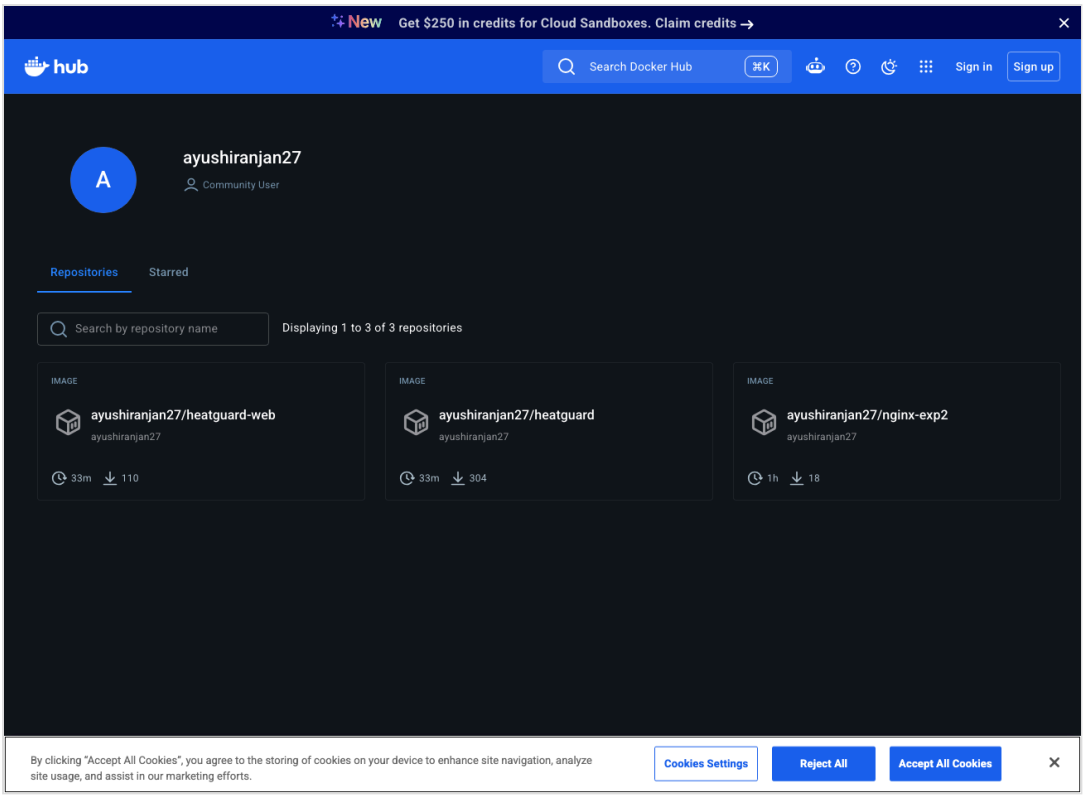


Figure 7. Docker Hub repositories of user ayushiranjan27 (captured after the v2 push).

Results. Backend image 136 MB and frontend image 22 MB (compressed, v2.0). Both containers report healthy; the frontend reaches the backend through its `/api/` proxy. Invalid input returns 404/422. With the backend stopped, the frontend stayed up and API calls returned HTTP 504 (gateway error) until the backend was restarted (HTTP 200). Docker Hub lists both images for linux/amd64 and linux/arm64; the pulled images ran healthy. Frontend 2.1 was pushed the same way for both platforms.

Conclusion. Both tiers were containerised, run together and published; anyone can start the full application with `docker compose up` on Intel or Apple Silicon machines.

Experiment 3: Application Deployment using Kubernetes

Aim (manual): deploy an application using Kubernetes, use kubectl commands and expose an app (preferably the image created in the Docker experiment). **Use-case connection:** both HeatGuard images from Docker Hub run as replicated, self-healing Deployments, as an early-warning service must stay available, including while it is updated.

Procedure. The manual used the Killercoda playground; this project used **kind** (Kubernetes-in-Docker), a real Kubernetes cluster running locally, so every step could be scripted (`experiments/exp3_kubernetes.sh`) and logged. The backend is a ClusterIP Service (internal only); the frontend is a NodePort Service on 30080 and finds the backend by its Kubernetes DNS name. Both Deployments use `maxSurge: 1`, `maxUnavailable: 0` and a 5-second `preStop` delay. The backend's CPU/memory limits are read from the Experiment 7 right-sizing decision.

k8s/api.yaml

```
# Experiment 3: HeatGuard backend API (7-day forecast). Image from Docker Hub (Experiment 2).
# Internal only: ClusterIP service, reached by the frontend through its /api/ proxy.
# Resource limits = right-sized values measured in Experiment 7.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: heatguard-api
  labels: {app: heatguard-api, tier: backend}
spec:
  replicas: 2
  selector:
    matchLabels: {app: heatguard-api}
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}   # never drop below the desired replica count
  template:
    metadata:
      labels: {app: heatguard-api, tier: backend}
    spec:
      terminationGracePeriodSeconds: 20
      containers:
        - name: api
          image: docker.io/DOCKERHUB_USER/heatguard:2.0
          imagePullPolicy: Always
          ports:
            - containerPort: 8000
          resources:
            requests: {cpu: 250m, memory: 128Mi}
            limits: {cpu: API_CPU_LIMIT, memory: API_MEM_LIMIT}
          readinessProbe:
            httpGet: {path: /health, port: 8000}
            initialDelaySeconds: 3
            periodSeconds: 5
          livenessProbe:
            httpGet: {path: /health, port: 8000}
            initialDelaySeconds: 10
            periodSeconds: 10
          lifecycle:
            preStop:
              exec: {command: ["sleep", "5"]}   # let the Service remove this pod before it stops
---
apiVersion: v1
kind: Service
metadata:
  name: heatguard-api
spec:
  type: ClusterIP
  selector: {app: heatguard-api}
  ports:
    - port: 8000
      targetPort: 8000
```

k8s/web.yaml

```
# Experiment 3: HeatGuard web frontend (nginx). Serves the app and proxies /api/ to the backend.
# Exposed outside the cluster on NodePort 30080 (mapped to localhost:30080 by kind-config.yaml).
apiVersion: apps/v1
kind: Deployment
metadata:
  name: heatguard-web
  labels: {app: heatguard-web, tier: frontend}
spec:
  replicas: 2
  selector:
    matchLabels: {app: heatguard-web}
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  template:
    metadata:
      labels: {app: heatguard-web, tier: frontend}
    spec:
      terminationGracePeriodSeconds: 20
      containers:
        - name: web
          image: docker.io/DOCKERHUB_USER/heatguard-web:2.1
          imagePullPolicy: Always
          env:
            - name: API_URL
              value: http://heatguard-api:8000   # Kubernetes DNS name of the backend Service
          ports:
            - containerPort: 8080
          resources:
            requests: {cpu: 25m, memory: 32Mi}
            limits: {cpu: 200m, memory: 64Mi}
          readinessProbe:
            httpGet: {path: /healthz, port: 8080}
            initialDelaySeconds: 2
            periodSeconds: 5
          livenessProbe:
            httpGet: {path: /healthz, port: 8080}
            initialDelaySeconds: 5
            periodSeconds: 10
          lifecycle:
```

experiments/results/exp3_kubernetes.log

```
### 1. Deploy backend (ClusterIP) and frontend (NodePort) from Docker Hub
$ kubectl apply -f /tmp/heatguard-api.yaml -f /tmp/heatguard-web.yaml
deployment.apps/heatguard-api created
service/heatguard-api created
deployment.apps/heatguard-web created
service/heatguard-web created

$ kubectl rollout status deployment/heatguard-api --timeout=300s
Waiting for deployment spec update to be observed...
Waiting for deployment spec update to be observed...
Waiting for deployment "heatguard-api" rollout to finish: 0 out of 2 new replicas have been updated...
Waiting for deployment "heatguard-api" rollout to finish: 0 of 2 updated replicas are available...
Waiting for deployment "heatguard-api" rollout to finish: 1 of 2 updated replicas are available...
deployment "heatguard-api" successfully rolled out

$ kubectl rollout status deployment/heatguard-web --timeout=300s
deployment "heatguard-web" successfully rolled out

$ kubectl get deployments,pods,services -o wide
NAME                            READY   UP-TO-DATE   AVAILABLE   AGE   CONTAINERS   IMAGES
SELECTOR
deployment.apps/heatguard-api   2/2     2            2           67s   api          docker.io/ayushiranjan27/heat
guard:2.0       app=heatguard-api
deployment.apps/heatguard-web   2/2     2            2           67s   web          docker.io/ayushiranjan27/heat
guard-web:2.0   app=heatguard-web

NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE                      NO
MINATED NODE   READINESS GATES
pod/heatguard-api-8c955c984-82rsc   1/1     Running   0          61s   10.244.0.7   heatguard-control-plane   <n
one>           <none>
pod/heatguard-api-8c955c984-b7856   1/1     Running   0          61s   10.244.0.5   heatguard-control-plane   <n
one>           <none>
pod/heatguard-web-66cf47486d-gb7hk   1/1     Running   0          61s   10.244.0.8   heatguard-control-plane   <n
one>           <none>
pod/heatguard-web-66cf47486d-s4cr5   1/1     Running   0          61s   10.244.0.2   heatguard-control-plane   <n
one>           <none>

NAME                    TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE   SELECTOR
service/heatguard-api   ClusterIP   10.96.212.217   <none>        8000/TCP       67s   app=heatguard-api
service/heatguard-web   NodePort    10.96.57.17     <none>        80:30080/TCP   67s   app=heatguard-web
service/kubernetes      ClusterIP   10.96.0.1       <none>        443/TCP        71s   <none>

$ kubectl describe pod heatguard-api-8c955c984-82rsc | grep -E 'Image:|Limits|Requests|cpu|memory|Pulled|preStop'
    Image:          docker.io/ayushiranjan27/heatguard:2.0
    Limits:
      cpu:     1250m
```

experiments/results/exp3_kubernetes.log

```
### 2. Service access: browser -> NodePort 30080 -> frontend -> heatguard-api Service -> backend pods
$ curl -s http://localhost:30080/healthz
ok

# backend pod answering each request (load-balanced by the heatguard-api Service):
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"heatguard-api-8c955c984-82rsc"}
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"heatguard-api-8c955c984-82rsc"}
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"heatguard-api-8c955c984-82rsc"}
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"heatguard-api-8c955c984-b7856"}
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"heatguard-api-8c955c984-b7856"}
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"heatguard-api-8c955c984-b7856"}
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"heatguard-api-8c955c984-b7856"}
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"heatguard-api-8c955c984-82rsc"}

$ curl -s -X POST http://localhost:30080/api/predict   (Delhi, real IMD values 21-27 May 2024)
highest alert: Heatwave | validated: Heatwave | first alert day: 1 | served by heatguard-api-8c955c984-b7856 in
8.32 ms
  day 1 2024-05-28   44.1 C (+/-0.888)  alert basis  45.3 C  Heatwave         validated
  day 2 2024-05-29   43.2 C (+/-1.299)  alert basis  45.9 C  Heatwave         validated
  day 3 2024-05-30   42.5 C (+/-1.515)  alert basis  45.5 C  Heatwave         validated
  day 4 2024-05-31   42.1 C (+/-1.645)  alert basis  45.4 C  Heatwave         validated
  day 5 2024-06-01   42.0 C (+/-1.715)  alert basis  45.7 C  Heatwave         low
  day 6 2024-06-02   41.9 C (+/-1.761)  alert basis  45.1 C  Heatwave         low
  day 7 2024-06-03   41.8 C (+/-1.79)  alert basis  45.3 C  Heatwave         low

# invalid input through the cluster
{"detail":"unknown city 'Atlantis'. Supported: Delhi, Jaipur, Lucknow, Patna, Ahmedabad, Nagpur, Kolkata, Bhubane
swar, Hyderabad, Chennai"} HTTP 404

# the backend is not exposed outside the cluster (type ClusterIP, no NodePort):
$ kubectl get service heatguard-api -o jsonpath={.spec.type} {.spec.ports}
ClusterIP [{"port":8000,"protocol":"TCP","targetPort":8000}]
```

experiments/results/exp3_kubernetes.log

```
### 3. Horizontal scaling: frontend 2 -> 4, backend 2 -> 3
$ kubectl scale deployment heatguard-web --replicas=4
deployment.apps/heatguard-web scaled

$ kubectl scale deployment heatguard-api --replicas=3
deployment.apps/heatguard-api scaled

$ kubectl rollout status deployment/heatguard-web --timeout=120s
Waiting for deployment "heatguard-web" rollout to finish: 2 of 4 updated replicas are available...
Waiting for deployment "heatguard-web" rollout to finish: 3 of 4 updated replicas are available...
deployment "heatguard-web" successfully rolled out

$ kubectl rollout status deployment/heatguard-api --timeout=120s
Waiting for deployment "heatguard-api" rollout to finish: 2 of 3 updated replicas are available...
deployment "heatguard-api" successfully rolled out

$ kubectl get pods -o wide
NAME                             READY   STATUS    RESTARTS   AGE   IP           NODE                       NOMIN
ATED NODE   READINESS GATES
heatguard-api-8c955c984-7kx69    1/1     Running   0          11s   10.244.0.11   heatguard-control-plane   <none
>           <none>
heatguard-api-8c955c984-82rsc    1/1     Running   0          73s   10.244.0.7    heatguard-control-plane   <none
>           <none>
heatguard-api-8c955c984-b7856    1/1     Running   0          73s   10.244.0.5    heatguard-control-plane   <none
>           <none>
heatguard-web-66cf47486d-8gml2   1/1     Running   0          11s   10.244.0.10   heatguard-control-plane   <none
>           <none>
heatguard-web-66cf47486d-8j8qb   1/1     Running   0          11s   10.244.0.9    heatguard-control-plane   <none
>           <none>
heatguard-web-66cf47486d-gb7hk   1/1     Running   0          73s   10.244.0.8    heatguard-control-plane   <none
>           <none>
heatguard-web-66cf47486d-s4cr5   1/1     Running   0          73s   10.244.0.2    heatguard-control-plane   <none
>           <none>
```

experiments/results/exp3_kubernetes.log

```
### 4. Self-healing: delete a backend pod; the Deployment replaces it
$ kubectl delete pod heatguard-api-8c955c984-7kx69 --wait=false
pod "heatguard-api-8c955c984-7kx69" deleted from default namespace

$ kubectl get pods -l app=heatguard-api
NAME                             READY   STATUS              RESTARTS   AGE
heatguard-api-8c955c984-7kx69    1/1     Terminating         0          11s
heatguard-api-8c955c984-82rsc    1/1     Running             0          73s
heatguard-api-8c955c984-b7856    1/1     Running             0          73s
heatguard-api-8c955c984-jj74h    0/1     ContainerCreating   0          0s

$ kubectl get pods -l app=heatguard-api
NAME                             READY   STATUS    RESTARTS   AGE
heatguard-api-8c955c984-82rsc    1/1     Running   0          88s
heatguard-api-8c955c984-b7856    1/1     Running   0          88s
heatguard-api-8c955c984-jj74h    1/1     Running   0          15s

$ kubectl get deployment heatguard-api
NAME            READY   UP-TO-DATE   AVAILABLE   AGE
heatguard-api   3/3     3            3           94s
```

experiments/results/exp3_kubernetes.log

```
### 5. Failure scenario: bad backend image -> rollout blocked, old pods keep serving -> rollback
$ kubectl set image deployment/heatguard-api api=docker.io/ayushiranjan27/heatguard:does-not-exist
deployment.apps/heatguard-api image updated

$ kubectl get pods -l app=heatguard-api
NAME                             READY   STATUS             RESTARTS   AGE
heatguard-api-6b55ff974c-5dl96   0/1     ImagePullBackOff   0          26s
heatguard-api-8c955c984-82rsc    1/1     Running            0          114s
heatguard-api-8c955c984-b7856    1/1     Running            0          114s
heatguard-api-8c955c984-jj74h    1/1     Running            0          41s

$ curl -s http://localhost:30080/api/health
{"status":"ok","version":"2.0","model":"ridge_regression","horizons":7,"pod":"heatguard-api-8c955c984-b7856"}

$ kubectl rollout undo deployment/heatguard-api
Warning: resource deployments/heatguard-api was previously managed with 'kubectl apply'. Rolling back will not up
date the kubectl.kubernetes.io/last-a
deployment.apps/heatguard-api rolled back

$ kubectl rollout status deployment/heatguard-api --timeout=180s
Waiting for deployment spec update to be observed...
Waiting for deployment "heatguard-api" rollout to finish: 1 old replicas are pending termination...
deployment "heatguard-api" successfully rolled out

$ kubectl get pods -l app=heatguard-api
NAME                             READY   STATUS        RESTARTS   AGE
heatguard-api-6b55ff974c-5dl96   0/1     Terminating   0          26s
```

```
experiments/results/exp3_kubernetes.log

### 6. Rolling update of the backend with continuous availability check
# strategy maxSurge=1, maxUnavailable=0 and preStop sleep 5 (v1 without preStop: 39/40 OK)
$ kubectl rollout restart deployment/heatguard-api && kubectl rollout status deployment/heatguard-api
Waiting for deployment "heatguard-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "heatguard-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "heatguard-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "heatguard-api" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "heatguard-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "heatguard-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "heatguard-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "heatguard-api" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "heatguard-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "heatguard-api" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "heatguard-api" rollout to finish: 1 old replicas are pending termination...
deployment "heatguard-api" successfully rolled out
# availability: 89/89 requests returned HTTP 200 (failed: 0)

$ kubectl rollout restart deployment/heatguard-web && kubectl rollout status deployment/heatguard-web
Waiting for deployment "heatguard-web" rollout to finish: 1 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 1 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 1 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 2 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 2 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 2 out of 4 new replicas have been updated...
```

Frontend release 2.0 → 2.1 on the running cluster (`experiments/release_frontend.sh`), with the landing page and `/api/health` polled every 0.25 s during the rollout:

experiments/results/frontend_2.1_release.log

```
### 2. Rolling update on Kubernetes with availability check (every 0.25 s through NodePort 30080)
$ kubectl get deployment heatguard-web -o wide
NAME            READY   UP-TO-DATE   AVAILABLE   AGE    CONTAINERS   IMAGES
SELECTOR
heatguard-web   4/4     4            4           121m   web          docker.io/ayushiranjan27/heatguard-web:2.0
app=heatguard-web

$ kubectl set image deployment/heatguard-web web=docker.io/ayushiranjan27/heatguard-web:2.1
deployment.apps/heatguard-web image updated

$ kubectl rollout status deployment/heatguard-web --timeout=300s
Waiting for deployment spec update to be observed...
Waiting for deployment "heatguard-web" rollout to finish: 0 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 1 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 1 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 1 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 2 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 2 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 2 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 2 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 3 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 3 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 3 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 3 out of 4 new replicas have been updated...
Waiting for deployment "heatguard-web" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "heatguard-web" rollout to finish: 1 old replicas are pending termination...
deployment "heatguard-web" successfully rolled out

# availability: 248/248 requests (landing page + /api/health) returned HTTP 200 (failed: 0)

$ kubectl get deployment heatguard-web -o wide
NAME            READY   UP-TO-DATE   AVAILABLE   AGE    CONTAINERS   IMAGES
SELECTOR
heatguard-web   4/4     4            4           122m   web          docker.io/ayushiranjan27/heatguard-web:2.1
app=heatguard-web

$ kubectl get pods -l app=heatguard-web
NAME                             READY   STATUS        RESTARTS   AGE
```

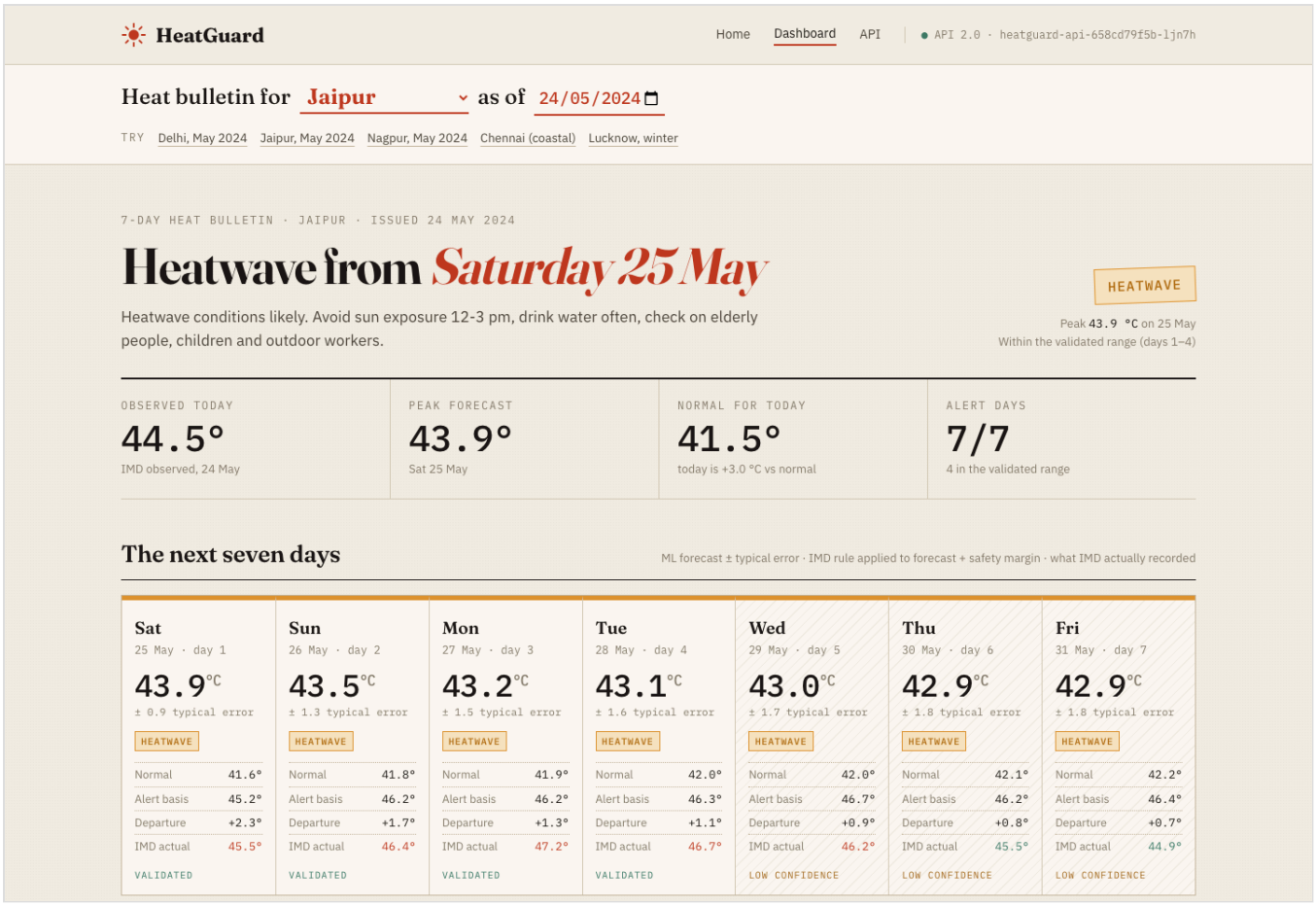


Figure 8. Dashboard (frontend 2.1) served through the Kubernetes NodePort (localhost:30080); the status label at the top right shows the backend pod that answered.

Results. Both images were pulled from Docker Hub; all pods became Ready; the frontend reached the backend through the `heatguard-api` Service, with requests spread across backend pods; scaling (frontend 2→4, backend 2→3) worked; a deleted backend pod was replaced automatically; a non-existent image tag stopped in `ImagePullBackOff` while existing pods kept serving, and `kubectl rollout undo` restored the Deployment. Rolling updates with continuous checks every 0.25 s: 89/89 successful during the backend rollout; 114/114 successful during the frontend rollout. The later frontend release to 2.1 served 248/248 requests successfully. No request failed, so the preStop delay removed the single failure observed in v1 (39/40).

Conclusion. Kubernetes provided declarative deployment, service discovery between the two tiers, scaling, self-healing, safe rollback and rolling updates for HeatGuard.

Experiment 7: Cloud Cost Optimization through Resource Right-Sizing

Aim (manual): analyse CPU and memory utilisation of an over-provisioned workload, apply right-sizing and measure cost and efficiency improvements without performance degradation. **Use-case connection:** the workload is the HeatGuard v2 backend itself, answering 7-day forecast requests built from real 2024 IMD data.

Procedure. (1) Run `ayushiranjan27/heatguard:2.0` with 2 vCPU / 2 GB (`docker run --cpus 2 --memory 2g`). (2) Send the fixed workload: 15,000 `POST /predict` requests, 20 concurrent clients, 100 warm-up requests, fixed seed (`loadtest/loadtest.py`), while sampling `docker stats` about once per second. (3) Repeat 3 times. (4) Right-size with the manual's rule, "slightly above peak with a safety buffer"

(`experiments/rightsize.py`): CPU = peak × 1.25 rounded up to 0.25 vCPU; memory = peak × 1.5 rounded up to 64 MiB. (5) Re-run the same workload 3 times with the new limits. (6) Estimate cost.

Right-sizing decision: observed peak 0.868 cores and 118.2 MiB → **1.25 vCPU / 192m**. These values are used as the backend limits in Kubernetes (Exp 3).

Configuration	CPU peak (% of 1 core)	CPU used (% of limit)	Memory peak (MiB)	Memory used (% of limit)	Throughput (req/s)	p50 latency (ms)	p95 latency (ms)	Errors	Est. USD / hour	Est. USD / 1M predictions
over-provisioned 2.0 vCPU / 2g	85.7 ± 1.3	42.8 ± 0.7	113.9 ± 6.9	5.6 ± 0.3	416.8 ± 13.6	21.6 ± 2.1	162.1 ± 5.6	0	0.0529	0.0353
right-sized 1.25 vCPU / 192m	99.1 ± 3.6	79.3 ± 2.9	106.5 ± 1.3	55.5 ± 0.7	436.4 ± 18.4	26.3 ± 3.2	152.1 ± 6.8	0	0.0303	0.0193

Change after right-sizing: throughput +4.7%, p50 latency +21.8%, p95 latency -6.2%, estimated hourly cost -42.7%, estimated cost per million predictions -45.3%. Errors: 0 and 0; out-of-memory kills: none.

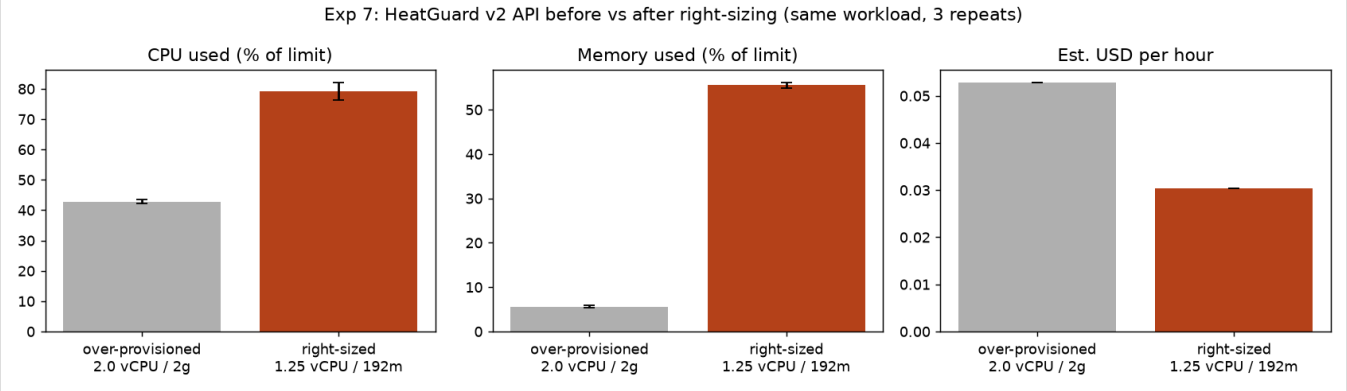


Figure 9. Experiment 7: utilisation of the allocated resources and estimated hourly cost before and after right-sizing (mean ± std, 3 runs).

Cost basis (estimate, not an actual charge): AWS Fargate on-demand, Linux/ARM, Asia Pacific (Mumbai): USD 0.02383 per vCPU-hour and USD 0.00261 per GB-hour, from the AWS public price list (publication date 2026-09-11), saved as `experiments/pricing/aws_ecs_ap-south-1_index.json`. Hourly cost = vCPU × 0.02383 + GB × 0.00261; cost per million predictions = hourly cost ÷ (throughput × 3600) × 10^6. No cloud account was used.

Conclusion. The backend used under half of the 2 allocated CPUs (42.8% of the limit) and about 6% of its memory; a single Uvicorn process cannot use more than about one core, so the extra CPU was idle. After right-sizing, throughput changed by +4.7% and p95 latency by -6.2%, but median (p50) latency rose by +21.8% (21.6 → 26.3 ms), so the manual's "without performance degradation" holds for throughput and tail latency but not fully for median latency. Given the -42.7% estimated cost change, this trade-off is acceptable for this service.

Experiment 8: FinOps Principles and Instance-Type Comparison

Aim (manual): compare different instance (resource) configurations for the same workload and identify the most cost-efficient option without performance degradation. **Use-case connection:** chooses the cheapest

size on which the heatwave API still meets its service target.

Procedure (manual algorithm steps 1–10). Workload identical to Experiment 7. Candidates: three AWS Fargate-valid sizes, emulated with Docker limits. The **performance target was written down before running** (`experiments/exp8_configs.json`), using the rule: At most 25% lower throughput and at most 50% higher p95 latency than the over-provisioned reference (2 vCPU / 2 GB: ~417 req/s, p95 ~162 ms), no errors, no OOM kill. This gives throughput ≥ 300 req/s, p95 latency ≤ 250 ms, zero errors and no OOM kill. Each configuration ran 3 times.

Configuration	CPU peak (% of 1 core)	CPU used (% of limit)	Memory peak (MiB)	Memory used (% of limit)	Throughput (req/s)	p50 latency (ms)	p95 latency (ms)	Errors	Est. USD / hour	Est. USD / 1M predictions	Meets target
low 0.25 vCPU / 512m	27.1 ± 0.1	108.4 ± 0.4	119.7 ± 0.3	23.4 ± 0.1	62.9 ± 0.8	289.4 ± 2.5	869.6 ± 49.2	0	0.0073	0.0320	no
medium 0.5 vCPU / 1g	50.8 ± 0.6	101.7 ± 1.1	118.3 ± 0.8	11.6 ± 0.1	302.7 ± 8.3	76.3 ± 1.1	109.2 ± 2.9	0	0.0145	0.0133	yes
high 1.0 vCPU / 2g	83.1 ± 5.9	83.1 ± 5.9	114.5 ± 7.2	5.6 ± 0.3	435.7 ± 7.3	22.2 ± 1.4	153.0 ± 3.1	0	0.0290	0.0185	yes

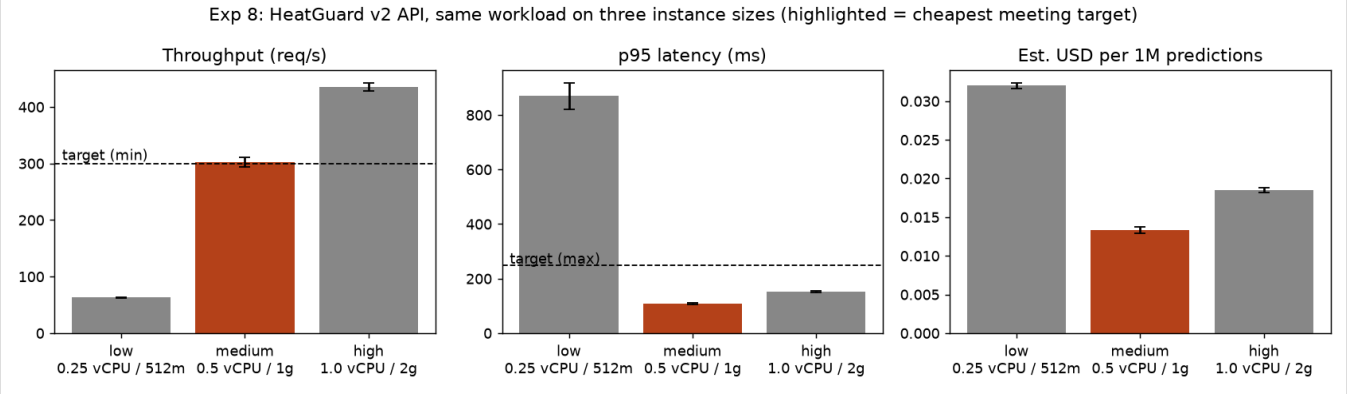


Figure 10. Experiment 8: throughput, p95 latency (dashed lines = targets) and estimated cost per million predictions (mean ± std, 3 runs).

Result. The most cost-efficient configuration meeting the target is **medium (0.5 vCPU / 1g)** at an estimated USD 0.0133 per million predictions. It passes on the mean of 3 runs, but only 2 of 3 individual runs reached 300 req/s (runs: 295.5, 300.7, 311.8 req/s), so it is a borderline choice with little headroom. low (0.25 vCPU) failed: throughput 62.9 < 300 req/s, p95 869.6 > 250 ms. At 0.25 vCPU the backend was CPU-saturated; its Docker health check timed out under load (the container was reported "unhealthy"), which in Kubernetes would trigger liveness-probe restarts.

Conclusion. The cheapest size per hour (low) was the most expensive per prediction, because it served so few requests; cost must be judged per unit of useful work. Moving from medium to high (1.0 vCPU) raised throughput by +43.9% at +39.1% cost per prediction; it would be the safer choice if more headroom above the target were wanted. Unlike v1, the target was defined with both a throughput and a latency requirement, closing the gap noted in the v1 analysis.

7. Testing and results

Automated tests (`pytest -v`, result: **18 passed in 2.38s**):

Test	Type
test_health_reports_loaded_model	representative / rules
test_cities_list	representative / rules
test_observed_returns_real_imd_window	representative / rules
test_observed_history_for_charts	representative / rules
test_predict_returns_7_consecutive_days	representative / rules
test_predict_hot_spell_raises_alert	representative / rules
test_predict_winter_is_normal	representative / rules
test_forecast_close_to_actual_on_real_day	representative / rules
test_outlook_covers_all_cities_sorted_by_risk	representative / rules
test_model_info_has_7_horizons	representative / rules
test_unknown_city_404	invalid input / failure
test_wrong_number_of_values_422	invalid input / failure
test_implausible_temperature_422	invalid input / failure
test_non_numeric_and_bad_date_422	invalid input / failure
test_observed_outside_data_range_404	invalid input / failure
test_observed_history_too_long_422	invalid input / failure
test_severity_rules	representative / rules
test_day_of_year_aligns_leap_and_non_leap	representative / rules

Failure scenarios demonstrated on the running system: unknown city (404), 3 values instead of 7 (422), IMD missing-value code 99.9 °C (422), invalid date (422), date outside the data (404), backend stopped behind the frontend (gateway error, recovered on restart), non-existent image tag in Kubernetes (ImagePullBackOff, rolled back), deleted pod (recreated). In the frontend, rejected what-if values are shown as an error message.

8. Requirement-to-test checklist

Requirement (assignment brief)	Implementation	Evidence	Status
Working prototype: input → processing → output	Web frontend + API: city/date (or 7 typed values) → 7-day forecast, alerts, advice, map	exp2_docker.log, screenshots 01–04	Verified
AI/ML component appropriate to use case	7 ridge-regression models (one per forecast day) on IMD data, compared with GBM and 2 baselines	models/metrics.json, train_output.txt	Verified
Rule-based part identified	IMD heatwave criteria (app/severity.py), labelled rule-based	test_severity_rules	Verified
Real / public dataset documented	IMD Pune 1° gridded Tmax 2014–2024, download script	data/processed/city_tmax.csv	Verified
Labelled architecture diagram	docs/architecture.html → 00_architecture_diagram.png	Figure 1	Verified
Exp 2: containerise, run, push to Docker Hub	Backend + frontend images, Compose, multi-arch push, pull-and-run check	exp2_docker.log; archive_v1/exp2_manual_tasks.log (nginx, hello-world, ubuntu)	Verified
Exp 3: deploy, expose, pod status, scaling/recovery	kind: 2 Deployments, ClusterIP + NodePort, scaling, self-healing, bad-image rollback, rolling update with availability check	exp3_kubernetes.log, screenshot 05	Verified
Exp 7: measure CPU/memory, right-size, compare	docker stats under fixed load, rule-based right-sizing, 3 repeats each	exp7/*, rightsizing_decision.json	Verified
Exp 8: same workload on configs, cheapest meeting target	3 Fargate-valid sizes, target fixed before the runs	exp8/*, exp8_configs.json, summary.md	Verified
Exp 7 & 8 same workload, cost basis stated	Same loadtest.py, seed, 15,000 requests, 20 clients; AWS price list saved	pricing/aws_ecs_ap-south-1_index.json	Verified
Representative inputs tested	pytest + curl on real IMD windows; frontend scenarios (Delhi, Jaipur, Nagpur, Chennai, Lucknow)	pytest_output.txt, exp2_docker.log	Verified
≥1 invalid-input / failure scenario	unknown city 404, wrong length 422, 99.9 °C 422, bad date 422, date outside data 404, backend down, bad image tag, deleted pod	pytest_output.txt, exp2_docker.log, exp3_kubernetes.log	Verified
Measurements (error, latency, throughput, CPU/mem, cost)	MAE per horizon, alert recall/precision, req/s, p50/p95, docker stats, est. USD	metrics.json, summary.md	Verified
Clean-environment setup	v1: fresh clone + venv + tests + retrain reproduced results; v2: images pulled from Docker Hub and run with Compose. Full README not repeated on a second machine	exp2_docker.log §5	Partially Verified
Demonstration video (5 min)	Plan in docs/DEMO_PLAN.md	link: to be added	Not Verified

9. Limitations, data privacy and improvements

- **Spatial resolution:** 1° grid cells (~100 km) are averages, not station readings.
- **Forecast inputs:** temperature history and season only (no humidity, wind or numerical weather model); skill at days 5–7 is close to climatology.
- **Alerts:** per cell per day; IMD declares heatwaves when criteria hold at ≥ 2 stations of a sub-division for 2 consecutive days. Normals come from 2014–2022, not 30-year station normals. Margins were tuned on a validation year with only 14 heatwave city-days; precision is low at longer horizons.
- **Historical replay:** the dashboard forecasts from any date in 2014–2024 using IMD archives; it is not connected to a live data feed.
- **Experiments 7 and 8:** instance sizes were emulated with Docker limits on one laptop; the single Python load generator limits throughput; costs are estimates.
- **Experiment 3:** single-node local cluster, not a cloud VM.
- **Data privacy:** only public, non-personal weather data; no user data stored; both containers run as non-root users; the backend is not exposed outside the cluster; no credentials in the repository.
- **Improvements:** humidity/heat-index inputs and IMD forecast products; consecutive-day rule; calibrated uncertainty (quantile regression); scheduled data refresh; LLM multilingual advisories with human review (use case §13); Horizontal Pod Autoscaler; repeat Experiments 7–8 on real cloud instances.

10. Version history

v1.0/1.1: single FastAPI container with a built-in dashboard and a next-day forecast; v1.1 corrected the coastal heatwave rule. **v2.0**: 7-day forecast, separate backend and frontend images, two-tier Kubernetes deployment with zero-downtime settings. **Frontend 2.1** (current): new landing page and redesigned dashboard; frontend-only change, released with a measured rolling update. Backend stays 2.0. All v1 experiment logs and measurements are kept unchanged in `experiments/results/archive_v1/`; every v2 experiment was re-run with the same scripts.

11. Conclusion

HeatGuard turns real IMD observations into 7-day maximum-temperature forecasts and IMD-style heatwave alerts, with each day's reliability measured and shown to the user. The same two images were containerised and published (Exp 2), operated on Kubernetes with scaling, recovery and rolling updates (Exp 3), right-sized from measured usage (Exp 7) and sized against an explicit service target (Exp 8). Each experiment acts on the same artifacts, so together they form one pipeline from data and model to a cost-aware, resilient deployment.

12. References

1. Somaiya Vidyavihar University, AI Use Case KJS-CES-01: Climate Intelligence for Heatwave Monitoring, Prediction, and Early Warning (use-case document).
2. Somaiya Vidyavihar University, Cloud Computing Experiment manuals 2, 3, 7 and 8 (KJSSE, 2025–26).
3. IMD Pune, High resolution daily gridded maximum temperature data (1°×1°), https://www.imdpune.gov.in/cmpg/Griddata/Max_1_Bin.html
4. A. K. Srivastava, M. Rajeevan, S. R. Kshirsagar, "Development of a high resolution daily gridded temperature data set (1969–2005) for the Indian region", Atmospheric Science Letters, 10(4), 249–254, 2009.

5. India Meteorological Department, Heat wave definitions, https://mausam.imd.gov.in/pdfs/heatcolduser/Definition.pdf
6. Amazon Web Services, AWS Price List (AmazonECS, ap-south-1), publication 2026-09-11, https://pricing.us-east-1.amazonaws.com/offers/v1.0/aws/AmazonECS/current/ap-south-1/index.json
7. Docker documentation, https://docs.docker.com/ ; Kubernetes documentation, https://kubernetes.io/docs/ ; kind, https://kind.sigs.k8s.io/
8. scikit-learn, https://scikit-learn.org/ ; FastAPI, https://fastapi.tiangolo.com/ ; nginx, https://nginx.org/
9. Chart.js, https://www.chartjs.org/ ; Leaflet, https://leafletjs.com/ ; Natural Earth (India point-of-view boundaries), https://www.naturalearthdata.com/
10. FinOps overview, Microsoft Learn, https://learn.microsoft.com/en-us/cloud-computing/finops/overview

13. Acknowledgement of AI-assisted work

This project was developed with the help of Claude (Anthropic), an AI assistant, used for analysing the assignment documents, writing code and scripts, and drafting this report. All code was executed on the student's machine; all results, logs and screenshots were produced by real runs and were checked by the student. The student created the Docker Hub account and performed the `docker login` herself. Command outputs in this report are copied verbatim from the log files in `experiments/results/` (Docker build progress lines omitted for length); screenshots of the frontend and Docker Hub were captured with headless Google Chrome from the running services.